AGENDA CRUD

Guía completa para instalar, configurar, ejecutar y utilizar la aplicación

Este manual describe el proyecto entregado en **crud_agenda_app.zip**. Está pensado para quien quiera poner la aplicación en marcha y para quien necesite entender su estructura y sus límites antes de ampliarla.

En pocas palabras
Es una aplicación web de una sola página construida con Python y Reflex. Permite crear, editar, completar y eliminar tareas. SQLAlchemy guarda los datos en una base SQLite local.

Qué encontrarás

- Requisitos y preparación del entorno.
- Instalación reproducible con `uv` y arranque local.
- Configuración disponible y ubicación efectiva del archivo `.env`.
- Operaciones de usuario con un ejemplo completo.
- Estructura del código, persistencia, solución de problemas y limitaciones observadas.

Ficha técnica

Elemento	Implementación observada
Interfaz y servidor	Reflex (aplicación Python con frontend web generado)
Persistencia	SQLAlchemy; SQLite por defecto
Validación	Pydantic: título, prioridad y fecha
Inicio de la aplicación	Inicializa tablas automáticamente al importar el módulo
Ruta de pantalla	Una página en `/`, título «Agenda»

Alcance. Esta guía se basa en los fuentes, configuración y archivos incluidos en el ZIP. El proyecto no incluye un README ni instrucciones propias de despliegue de producción. Los comandos aquí descritos son para desarrollo local.

1. Preparar el entorno

Antes de empezar, extrae el ZIP en una carpeta de trabajo. Los comandos siguientes suponen que la carpeta extraída se llama `crud_agenda_app` y que abres una terminal dentro de ella.

Requisitos

- Python **3.14 o posterior**, según `requires-python` en `pyproject.toml`.
- `uv`, para crear el entorno e instalar las versiones fijadas en `uv.lock`.
- Un navegador moderno. Para ejecutar la aplicación, el equipo debe poder abrir puertos locales; Reflex inicia servidor y frontend de desarrollo.

Comprobar versiones
Ejecuta `python3 --version` (o `python --version` en Windows) y `uv --version`. Si tu sistema no ofrece Python 3.14, instala esa versión y asegúrate de que `uv` la detecta.

Instalación

Desde la carpeta que contiene `pyproject.toml`:

```
cd ruta/a/crud_agenda_app
uv sync
```

`uv sync` lee las dependencias del proyecto y el archivo de bloqueo, crea el entorno virtual local (normalmente `.venv`) e instala Reflex, SQLAlchemy, Pydantic y sus dependencias. No hace falta ejecutar `reflex init`: el proyecto ya trae configuración y paquete de aplicación.

Arrancar y detener

```
uv run reflex run
```

Deja la terminal abierta mientras uses la aplicación. Reflex informa las direcciones locales al iniciar; abre la URL local que muestre (normalmente `http://localhost:3000`). El backend de Reflex usa por defecto `http://localhost:8000`. Para detener el servidor, vuelve a la terminal y pulsa `Ctrl+C`.

Primer arranque
El módulo de la aplicación ejecuta `init_db()` al cargar y crea las tablas que falten. En la configuración predeterminada, SQLite guarda la base como `crud_agenda_app.db` relativa al directorio de ejecución.

Si el comando no encuentra Reflex

Comprueba que ejecutaste `uv sync` desde la carpeta correcta y que existe `pyproject.toml`. Usa `uv run reflex run` para ejecutar el Reflex instalado dentro del entorno del proyecto.

2. Configurar y entender el proyecto

La aplicación usa `config.py` para leer ajustes mediante Pydantic Settings. Las variables de entorno del sistema pueden sobrescribir los valores por defecto.

Variable	Valor predeterminado	Efecto
`ENV`	`dev`	Si vale `prod`, exige `SECRET_KEY`. No selecciona por sí sola un modo de despliegue.
`APP_NAME`	`crud_agenda_app`	Nombre de aplicación Reflex.
`DEBUG`	`false`	Ajuste definido, sin uso directo visible en el código revisado.
`DATABASE_URL`	`sqlite:///crud_agenda_app.db`	URL de SQLAlchemy para la base de datos.
`SECRET_KEY`	Sin valor	Obligatoria cuando `ENV=prod`; no se observa uso adicional en la lógica de tareas.
`OPENAI_API_KEY`	Sin valor	Declarada, pero no se usa en esta aplicación.
`API_URL`	`http://localhost:8000`	URL de API configurada para Reflex.

Ubicación de .env: detalle importante

Aunque el ZIP contiene un archivo `.env` vacío en la carpeta del proyecto, `config.py` calcula la ruta del entorno como `Path(__file__).resolve().parent.parent`. Como `config.py` está en la raíz del proyecto, Pydantic busca el archivo `.env` **un nivel por encima** de esa raíz. Por tanto, un `.env` creado dentro de `crud_agenda_app/` no se lee mediante esa ruta.

Para la configuración inicial recomendada, usa variables de entorno en la terminal. Si prefieres archivo, coloca el `.env` en la carpeta padre del proyecto debido al comportamiento actual del código. No guardes secretos en repositorios o archivos compartidos.

```
macOS / Linux (solo para esta terminal):
export DATABASE_URL="sqlite:///crud_agenda_app.db"
export ENV="dev"
uv run reflex run

PowerShell:
$env:DATABASE_URL = "sqlite:///crud_agenda_app.db"
$env:ENV = "dev"
uv run reflex run
```

Cambiar la ruta de la base
Por ejemplo, `DATABASE_URL="sqlite:////ruta/absoluta/agenda.db"` en macOS/Linux usa una ruta absoluta. Asegura que la carpeta exista y que tengas permiso de escritura. Una URL PostgreSQL requiere instalar además su controlador; el ZIP no define ese controlador ni un servicio de base de datos.

Estructura del proyecto

El código está organizado por responsabilidades; la carpeta `.web/` contiene recursos frontend generados por Reflex y no suele editarse a mano.

Ruta	Responsabilidad
`crud_agenda_app/pages/index.py`	Construye la pantalla, filas de tarea y formulario modal.
`states/task_state.py`	Estado Reflex y eventos de crear, editar, completar y borrar.
`services/task_service.py`	Consultas y operaciones SQLAlchemy.
`db/models/task.py`	Modelo ORM de la tabla `tasks`.
`db/schemas/task.py`	DTO y validación con Pydantic.
`db/base.py`	Motor, sesiones e inicialización de tablas.
`config.py` y `rxconfig.py`	Ajustes del proyecto y configuración Reflex.

Recorrido de una operación

La página llama a eventos de `TaskState`; estos validan la entrada con los esquemas Pydantic y delegan persistencia a `task_service`. El servicio abre una sesión SQLAlchemy y consulta o modifica la tabla `tasks`. Al guardar, completar o borrar, el estado vuelve a cargar la lista desde la base.

Datos de una tarea

Cada registro tiene identificador, título, descripción, prioridad, fecha límite opcional, estado `done` y fecha de creación. La prioridad por defecto es media y el estado por defecto es pendiente. La fecha se valida como fecha de calendario; el título admite entre 1 y 200 caracteres.

Vista de arquitectura
Navegador → página Reflex → estado y eventos → servicio CRUD → SQLAlchemy → archivo SQLite. La página carga las tareas al abrirse; después de cada cambio, vuelve a consultar los registros para refrescar la lista.

3. Usar la agenda paso a paso

La pantalla principal muestra el encabezado «Agenda», el botón «New task» y las tareas existentes. La interfaz está en inglés en el código incluido.

Crear una tarea

- Pulsa **New task**.
- Escribe un título (obligatorio, máximo 200 caracteres).
- Añade una descripción si hace falta.
- Elige prioridad: `low`, `medium` o `high`. La opción inicial es `medium`.
- Selecciona una fecha límite opcional en el selector de fecha.
- Pulsa **Save**. Si el título no es válido o la prioridad no coincide, el formulario muestra un aviso de validación.

Ejemplo de tarea

Campo	Ejemplo
Título	Llamar al proveedor
Descripción	Confirmar disponibilidad para el pedido de noviembre.
Prioridad	high
Fecha límite	2026-11-10

Completar, editar o eliminar

- **Completar:** marca la casilla de la tarea. El título aparece tachado. Vuelve a marcarla para quitar el estado completado.
- **Editar:** pulsa el icono de lápiz; se abre el formulario con los valores actuales. Cambia los campos y guarda.
- **Eliminar:** pulsa el icono de papelera. La eliminación se ejecuta de inmediato y no se muestra una confirmación ni una función de deshacer.

Cómo se muestran los datos

- Las tareas se listan por fecha de creación descendente, con las más nuevas primero.
- La prioridad se representa con una etiqueta de color: alta roja, media ámbar y baja gris.
- La fecha se muestra cuando existe. La descripción se oculta si está vacía.
- Al recargar la página, la app vuelve a consultar la base de datos; las tareas guardadas permanecen allí.

Respaldo de los datos
Con SQLite, la información reside en un archivo local (por defecto `crud_agenda_app.db`). Detén la aplicación y copia ese archivo para crear una copia de seguridad. Si cambias `DATABASE_URL`, localiza y respalda la base del nuevo destino.

Qué no ofrece la interfaz

No hay búsqueda, filtros, categorías, vistas de calendario, exportación ni gestión de usuarios. Es una agenda sencilla de tareas, no un sistema multiusuario o un servicio desplegado.

4. Solución de problemas y límites

Síntoma	Qué revisar
`uv: command not found`	Instala uv siguiendo la documentación oficial de Astral y abre una terminal nueva.
Versión de Python incompatible	El proyecto exige Python 3.14+. Instala esa versión; revisa `python3 --version` y la detección de `uv`.
Error al resolver o instalar paquetes	Ejecuta `uv sync` desde la raíz del proyecto con conexión disponible. El proyecto fija dependencias en `uv.lock`.
El navegador no abre la app	Espera a que Reflex indique que el frontend está listo; usa la URL que imprime. Revisa si los puertos locales están ocupados o bloqueados.
Cambios no aparecen o estado raro	Recarga la página. Si el servidor muestra error, deténlo con Ctrl+C y vuelve a iniciar con `uv run reflex run`.
No encuentro los datos o se crea otra base	La ruta SQLite relativa depende del directorio actual de ejecución. Ejecuta desde la carpeta raíz del proyecto o define una URL absoluta con `DATABASE_URL`.
No carga el .env esperado	El código apunta al .env de la carpeta padre del proyecto. Define las variables en el entorno o coloca el archivo en la ubicación que realmente lee `config.py`.
Error al guardar una tarea	Comprueba que el título no esté vacío y no supere 200 caracteres; prioridad debe ser low, medium o high; fecha debe ser una fecha válida. Revisa también permisos de escritura de la base.

Limitaciones comprobadas en el código

- **Es un prototipo:** el propio código indica que la creación automática de tablas debería sustituirse por migraciones Alembic cuando evolucione el esquema.
- **Sin autenticación ni autorización:** no existe separación de tareas por usuario. No debe exponerse como servicio compartido tal cual.
- **Errores incompletos:** el formulario captura errores de validación de Pydantic; no se ve manejo de excepciones de base de datos. Los fallos de infraestructura pueden aparecer como error del servidor.
- **Operaciones simples:** borrar no pide confirmación; no hay papelera ni historial. No hay paginación ni filtrado.
- **Configuración sobrante:** DEBUG, OPENAI_API_KEY y SECRET_KEY están declaradas, pero su función no se refleja en el flujo CRUD; la clave solo se exige en ENV=prod.
- **Idioma:** etiquetas y mensajes de la interfaz están en inglés. Este documento está en español, pero la interfaz no está traducida.

Lista breve de comprobación

■ Python 3.14+ y uv disponibles ■ ZIP extraído ■ `uv sync` completado
■ `uv run reflex run` activo ■ URL local abierta ■ Tarea de prueba creada, editada y completada

Base de esta guía: revisión estática de `pyproject.toml`, `uv.lock`, `config.py`, `rxconfig.py` y los módulos de páginas, estado, servicios, modelos y esquemas incluidos en el ZIP. No se ejecutó ni modificó el proyecto fuente.